01. Plonky3 zkVM 개요

1.1 프로젝트 소개

이 프로젝트는 Plonky3 라이브러리를 사용하여 구현된 교육용 Zero-Knowledge Virtual Machine (zkVM)입니다. 스택 기반의 간단한 가상 머신을 실행하고, 그 실행의 정확성을 증명하는 STARK 증명을 생성합니다.

1.2 핵심 개념

1.2.1 Zero-Knowledge Proof (영지식 증명)

- 어떤 명제가 참임을 증명하되, 그 명제에 대한 정보는 전혀 노출하지 않는 암호학적 기법
- 이 구현체에서는 프로그램이 올바르게 실행되었음을 증명하되, 실행 과정의 세부 사항은 숨길 수 있음

1.2.2 STARK (Scalable Transparent ARgument of Knowledge)

- 확장 가능하고 투명한 영지식 증명 시스템
- Trusted Setup이 필요 없음 (투명성)
- 큰 계산에 대해서도 효율적으로 증명 생성 가능 (확장성)

1.2.3 zkVM (Zero-Knowledge Virtual Machine)

- 프로그램 실행을 검증 가능한 형태로 만드는 가상 머신
- 실행 추적(trace)을 생성하고, 이에 대한 영지식 증명을 생성

1.3 아키텍처 구성 요소

```
┌──────────────────────────────────────────────────────────────┐
| Plonky3 zkVM |
├──────────────────────────────────────────────────────────────┤
| |
| ┌──────────────┐ ┌──────────────┐ |
| | VM (가상머신) | ──> | Trace (추적) | |
| | | | | |
| | - 스택 실행 | | - 상태 기록 | |
| | - 명령어 처리 | | - 11개 컬럼 | |
| └──────────────┘ └──────────────┘ |
| | | |
| | ▼ |
| | ┌──────────────┐ |
| | | AIR (제약조건) | |
| | | | |
| | | - 상태 전이 | |
| | | - 다항식 제약 | |
| | └──────────────┘ |
| | | |
| | ▼ |
| | ┌──────────────┐ |
| └──────────────> | Prover (증명자)| |
```

```
│ │ │ │
│ │ - STARK 증명 │ │
│ │ - FRI 프로토콜 │ │
│ └──────────────┘ │
│ │ │
│ ▼ │
│ ┌──────────────┐ │
│ │Verifier(검증자)│ │
│ │ │ │
│ │ - 증명 검증 │ │
│ └──────────────┘ │
└──────────────────────────────────────────────────────────┘
```

1.4 주요 파일 구조

src/vm.rs

- 가상 머신 구현
- 스택 기반 명령어 실행
- 실행 추적 생성

src/p3.rs

- Plonky3 통합 코드
- AIR (Algebraic Intermediate Representation) 정의

- 증명 생성 및 검증 로직

src/lib.rs

- 테스트 코드
- 전체 시스템 통합

1.5 지원하는 명령어

이 zkVM은 5가지 기본 명령어를 지원합니다:

1. **Push(value)**: 값을 스택에 푸시
2. **Add**: 스택 상위 두 값을 더함
3. **Sub**: 스택 상위 두 값을 뺌 (두 번째 - 첫 번째)
4. **Mul**: 스택 상위 두 값을 곱함
5. **Div**: 스택 상위 두 값을 나눔 (두 번째 / 첫 번째)

1.6 실행 흐름

```
1. 프로그램 작성 (Instructions 벡터)
↓
2. VM 초기화 및 실행 (VM::new, VM::run)
↓
3. 실행 추적 생성 (VM::get_trace)
↓
4. AIR 제약 조건 정의 (VMAir::eval)
```

```
↓

5. STARK 증명 생성 (VMAir::generate_proof)

↓

6. 증명 검증 (verify)
```

1.7 사용된 Plonky3 라이브러리

- **p3-air**: AIR 정의를 위한 인터페이스
- **p3-field**: 유한체 연산 (Mersenne31 사용)
- **p3-matrix**: 행렬 연산 (추적 데이터 저장)
- **p3-uni-stark**: STARK 증명 생성 및 검증
- **p3-fri**: FRI (Fast Reed-Solomon Interactive Oracle Proofs) 프로토콜
- **p3-merkle-tree**: 머클 트리 기반 커밋먼트
- **p3-keccak**: Keccak 해시 함수

1.8 핵심 특징

1.8.1 스택 기반 실행

- 고정 크기 스택 (4개 요소)
- LIFO (Last In First Out) 구조
- 간단하고 검증 가능한 실행 모델

1.8.2 완전한 실행 추적

- 모든 명령어 실행 후 상태 기록

- 11개 컬럼으로 구성된 추적 테이블
- 패딩을 통한 2의 거듭제곱 크기 보장

1.8.3 대수적 제약 조건

- 각 명령어에 대한 다항식 제약
- 상태 전이의 정확성 보장
- 효율적인 검증 가능

이 문서는 plonky3 zkVM의 전체적인 개요를 제공합니다. 다음 문서들에서 각 구성 요소를 더 자세히 살펴보겠습니다.